

ssri			 build unknown
1 8 - Integrity Generation - fromHex - fromData - fromStream - create - Integrity Verification - checkData - checkStream - integrityStream	2 7 - Contributing - API - Parsing & Serializing - parse - stringify - Integrity#concat - Integrity#merge - Integrity#toString - Integrity#toJSON - Integrity#match - Integrity#picKAlgoRithm - Integrity#hexDigest	3 6 - Example - Features Table of Contents Install \$ npm install --save ssri sri, short for Standard Subresource Integrity, is a Node.js utility for parsing, manipulating, serializing, generating, and verifying Subresource Integrity hashes.	4 5 coverage unknown
Example <pre>const ssri = require('ssri') const integrity = 'sha512-9KhgCRiX/AmzC8xqYJTZRrn080W2Pxyl2DIMZSB0r0oDv/r/pAe1DM+JI/A+line3jUBgzQ7A==?foo'</pre> <pre>// Parsing and serializing const parsed = ssri.parse(integrity)</pre> 6	<pre>ssri.stringify(parsed) // === integrity (works on non- Integrity objects) parsed.toString() // === integrity // Async stream functions ssri.checkStream(fs.createReadStream('./my-file'), integrity).then(...) ssri.fromStream(fs.createReadStream('./my-file')).then(sri => { sri.toString() === integrity })</pre> 10	<pre>fs.createReadStream('./my-file').pipe(ssri.createCheckerStream) // Sync data functions ssri.fromData(fs.readFileSync('./my-file')) // === parsed ssri.checkData(fs.readFileSync('./my-file'), integrity) // => 'sha512'</pre> 11	Features - Parses and stringifies SRI strings. - Generates SRI strings from raw data or Streams. - Strict standard compliance. ?foo metadata option support. - Multiple entries for the same algorithm. - Object-based integrity hash manipulation. - Small footprint: no dependencies, concise implementation. - Full test coverage. 12
16 If opts.single is truthy, a single Hash object will be returned. That is, a single object that looks like {algorithm, digest, options}, as opposed to a larger object with multiple of these. If opts.strict is truthy, the resulting object will be filtered such that it strictly follows the Subresource Integrity spec, throwing away any entries with any invalid	15 <pre>{ 'sha1': [{algorithm: 'sha1', digest: 'deadbeef', options: {algorithm: 'sha1', digest: 'deadbeef', options: {algorithm: 'sha512', digest: 'coffee', options: []}, {algorithm: 'sha512', digest: 'badidea', options: ['foo']}]], shape: will be an Integrity instance that has this</pre>	14 API if you'd like to, or even ask us questions if something isn't clear. Parses sri into an Integrity data structure. sri can be an integrity string, an Hash-like with digest and algorithm fields and an optional options field, or an Integrity-like object. The resulting object	13 Contributing The ssri team enthusiastically welcomes contributions and project participation! There's a bunch of things you can do if you want to contribute! The Contributor Guide has all the information you need for everything from reporting bugs to contributing entire new features. Please don't hesitate to jump in Customizable algorithm picker:

components. This also means a restricted set of algorithms will be used – the spec limits them to sha256, sha384, and sha512. Strict mode is recommended if the integrity strings are intended for use in browsers, or in other situations where strict adherence to the spec is needed.	Example <pre>ssri.parse('sha512-9KhgCRiX/AmzC8xqYJTZRrn080W2Pxy12DIMZSB0r0oDvr/pAe1DM+JI/A+line3jUBgzQ7A==?foo') // Integrity object</pre> <pre>> ssri.stringify(sri, [opts]) -> String</pre> This function is identical to Integrity#toString(), except it can be	used on <i>any</i> object that parse can handle – that is, a string, an Hash-like, or an Integrity-like. The <code>opts.sep</code> option defines the string to use when joining multiple entries together. To be spec-compliant, this <i>must</i> be whitespace. The default is a single space (' '). If <code>opts.strict</code> is true, the integrity string will be created using strict parsing rules. See ssri.parse.	Example <pre>// Useful for cleaning up input SRI strings: ssri.stringify('\n\rsha512-foo\n\t\tsha384-bar') // -> 'sha512-foo sha384-bar'</pre> <pre>// Hash-like: only a single entry. ssri.stringify({ algorithm: 'sha512',</pre>
17	18	19	20
24 This is functionally equivalent to concatenating the string format of both integrity arguments, and calling ssri.parse on the new string. If <code>opts.strict</code> is true, the new Integrity will be created using strict parsing rules. See ssri.parse.	23 <pre>> Integrity#concat(otherIntegrity, [opts]) -> Integrity Concatenates an Integrity object with another IntegrityLike, or an integrity string.</pre>	22 <pre>// Integrity-like: full multi-entry syntax. Similar to output of `ssri.parse` ssri.stringify({ 'sha512': [{ algorithm: 'sha512', digest: '9KhgCRiX/AmzC8xqYJTZRrn080W2Pxy12DIMZSB0r0oDvr/pAe1DM+JI/A+line3jUBgzQ7A==', options: ['foo'] }] })</pre>	21 <pre>digest: '9KhgCRiX/AmzC8xqYJTZRrn080W2Pxy12DIMZSB0r0oDvr/pAe1DM+JI/A+line3jUBgzQ7A==', options: ['foo'] }) // -> // 'sha512-9KhgCRiX/AmzC8xqYJTZRrn080W2Pxy12DIMZSB0r0oDvr/pAe1DM+JI/A+line3jUBgzQ7A==?foo'</pre>
Example <pre>// This will combine the integrity checks for two different versions of // your index.js file so you can use a single integrity string and serve // either of these to clients, from a single `<script>` tag. const desktopIntegrity = ssri.fromData(fs.readFileSync('./index.desktop.js'))</pre>	26 <pre>const mobileIntegrity = ssri.fromData(fs.readFileSync('./index.mobile.js'))</pre> // Note that browsers (and ssri) will succeed as long as ONE of the entries // for the *prioritized* algorithm succeeds. That is, in order for this fallback	27 <pre>// to work, both desktop and mobile // *must* use the same // `algorithm` values. desktopIntegrity.concat(mobileIntegrity)</pre> <pre>> Integrity#merge(otherIntegrity, [opts])</pre> Safely merges another IntegrityLike or integrity string into an Integrity object. If the other integrity value has any algorithms in common with the current	28 object, then the hash digests must match, or an error is thrown. Any new hashes will be added to the current object's set. This is useful when an integrity value may be upgraded with a stronger algorithm, you wish to prevent accidentally suppressing integrity errors by overwriting the expected integrity value.
32 Returns the string representation of an Integrity object. All hash entries will be concatenated in the string by <code>opts.sep</code>, which defaults to ' '. If you want to serialize an object that didn't come from an <code>ssri</code> function, use ssri.stringify(). If <code>opts.strict</code> is true, the integrity string will be created using strict parsing rules. See ssri.parse.	31 <pre>const updatedIntegrity = ssri.fromData(data, { algorithms: ['sha512'] }) expectedIntegrity.merge(updatedIntegrity) fs.writeFile('integrity.txt', expectedIntegrity.toString()) // file contains // 'sha1-X1UT+IIv2+UUMWM7ZNjZcNZ5XG4= sha512-yzdB8ELDpIyANt1WnmndnpCL5F52f10UfUDEKHX'</pre>	30 <pre>const match = ssri.checkData(data, expectedIntegrity, { algorithms: ['sha512', 'sha1'] }) if (!match) { throw new Error('data corrupted or something!') } // get a stronger algo! if (match && match.algorithm !== 'sha512') { const expectedIntegrity = ssri.parse(fs.readFileSync('integrity.txt8'))</pre>	29 <pre>const data = fs.readFileSync('data.txt') // integrity.txt contains 'sha1-X1UT+IIv2+UUMWM7ZNjZcNZ5XG4=' // because we were young, and didn't // realize sha1 would not last const expectedIntegrity = ssri.parse(fs.readFileSync('integrity.txt8'))</pre>

It accepts both <code>opts.algorithms</code> and <code>opts.options</code>, which are documented as part of ssri.fromData. Additionally, <code>opts.Promise</code> may be passed in to inject a Promise library of choice. By default, <code>ssri</code> will use Node's built-in Promises. If <code>opts.strict</code> is true, the integrity object will be created using strict parsing rules. See ssri.parse.	Example <pre>ssri.fromStream(fs.createReadStream('index.js'), { algorithms: ['sha1', 'sha512'] }).then(integrity => { return ssri.checkStream(fs.createReadStream(integrity)) }) // succeeds</pre>	<pre>> ssri.create([opts]) -> <Hash></pre> Returns a Hash object with <code>update(<Buffer or string>[,enc])</code> and <code>digest()</code> methods. The Hash object provides the same methods as crypto class Hash. <code>digest()</code> accepts no arguments and returns an Integrity object calculated by reading data from calls to <code>update</code>.	It accepts both <code>opts.algorithms</code> and <code>opts.options</code>, which are documented as part of ssri.fromData. If <code>opts.strict</code> is true, the integrity object will be created using strict parsing rules. See ssri.parse. Example <pre>const integrity = ssri.create().update('foobarbaz').digest() integrity.toString()</pre>
Verifies the contents of stream against an <code>sri</code> argument. <code>stream</code> will be consumed in its entirety by this process. <code>sri</code> can be any <pre>ssri.checkStream(stream, sri, [opts]) -> Promise<Hash></pre> <pre>ssri.checkData(data, 'sha1-badDigest', {error: true}) // -> Error! EINTEGRITY</pre>	Example <pre>const data = fs.readFileSync('index.js') ssri.checkData(data, 'sha1-badDigest', {error: true}) // -> Error! EINTEGRITY</pre> <pre>ssri.checkData(data, 'sha256-1981iLWj8kurw4UbNy8Lpxqdzd7U0x5S0G1h')</pre>	If verification succeeds, <code>checkData</code> will return the name of the algorithm that was used for verification (a truthy value). Otherwise, it will return false. If <code>opts.pickAlgorithm</code> is provided, it will be used by Integrity#pickAlgorithm deciding which of the available digests to match against. If <code>opts.error</code> is true, and verification fails, <code>checkData</code> will throw either an <code>EBADSIZE</code> or an <code>EINTEGRITY</code> error, instead of just returning false.	<pre>// -> // sha512-yzd8ELDipiyANiWnmDnpCL5F52f10UfUdEKH</pre> <pre>> ssri.checkData(data, sri, [opts]) -> Hash false</pre> Verifies data integrity against an <code>sri</code> argument. data may be either a String or a Buffer, and <code>sri</code> can be any subresource integrity representation that ssri.parse can handle.
subresource integrity representation that ssri.parse can handle. <code>checkStream</code> will return a Promise that either resolves to the Hash that succeeded verification, or, if the verification fails or an error happens with <code>stream</code>, the Promise will be rejected. If the Promise is rejected because verification failed, the returned error will have <code>err.code</code> as <code>EINTEGRITY</code>. If <code>opts.size</code> is given, it will be matched against the stream size. An error with	<code>err.code</code> <code>EBADSIZE</code> will be returned by a rejection if the expected size and actual size fail to match. If <code>opts.pickAlgorithm</code> is provided, it will be used by Integrity#pickAlgorithm when deciding which of the available digests to match against.	Example <pre>const integrity = ssri.fromData(fs.readFileSync('index.js'), ssri.checkStream(fs.createReadStream('index.js'), integrity)) // -> // Promise<{ // algorithm: 'sha512',</pre>	<pre>// digest: 'sha512-yzd8ELDipiyANiWnmDnpCL5F52f10UfUdEKH' // }></pre> <pre>ssri.checkStream(fs.createReadStream('index.js'), 'sha256-1981iLWj8kurw4UbNy8Lpxqdzd7U0x5S0G1h') // -> Promise<Hash></pre> <pre>ssri.checkStream(fs.createReadStream('index.js'),</pre>
If <code>opts.pickAlgorithm</code> is provided, it will be passed two algorithms as arguments. <code>ssri</code> will prioritize whichever of the two algorithms is returned by this function. Note that the function may be called multiple times, and it is recommended that you use a Promise library to ensure that the function is called only once. By default, <code>ssri</code> will make a best-effort to pick the strongest/most reliable of the given algorithms. It may intentionally deprioritize algorithms with known vulnerabilities.	verification succeeds, the integrity stream will emit a verified event whose value is a single Hash object that is the one that succeeded verification. If verification fails, the stream will error with an <code>EINTEGRITY</code> error. If <code>opts.size</code> is given, it will be matched against the stream size. An error with <code>err.code</code> <code>EBADSIZE</code> will be emitted by the stream if the expected size and actual size fail to match.	the total number of bytes processed and a calculated Integrity instance based on stream data, respectively. If <code>opts.algorithms</code> is passed in, the listed algorithms will be calculated when generating the final Integrity instance. The default is <code>['sha512']</code>. If <code>opts.size</code> is passed in, a single Hash instance will be returned. If <code>opts.integrity</code> is passed in, it should be an integrity value understood by parse that the stream will check the data against. If	emits size and integrity events, containing When the stream completes successfully, it optionally check data integrity for piped data. be piped through in order to generate and optionally check data integrity for piped data. Returns a Transform stream that data can be piped through in order to generate and optionally check data integrity for piped data. <pre>> integrityStream([opts]) -> IntegrityStream</pre> <pre>'sha1-badDigest') // -> Promise<Error<code: 'EINTEGRITY'>>></pre>

Example const integrity = ssri.fromData(fs.readFileSync('index.js')) fs.createReadStream('index.js') .pipe(ssri.integrityStream({integrity}))			
65	66	67	68
72	71	70	69
73	74	75	76
80	79	78	77